

Proceso Unificado Rational

RUP

Seis mejores practicas

Análisis de Sistemas

MSc. Marco Coral

RATIONAL UNIFIED PROCESS (RUP) - 2003



Objetivos:

- ✓ Describir las 6 mejores prácticas en desarrollo de software.
- ✓ Describir el Rational Unified Process en términos de sus fases y disciplinas.
- ✓ Describir el modelo iterativo para desarrollo de software dirigido por riesgos (risk-driven)
- ✓ Comprender los fundamentos del proceso de personalización del RUP a las necesidades específicas de una organización.

Problemas en el desarrollo

- Necesidades no satisfechas
- Requerimientos revueltos
- Calidad pobre
- Módulos no aptos
- Descubrimiento tardío de errores
- Rendimiento inaceptable
- Falta de coordinación en el equipo
- Un desconfiable proceso de construcción-distribución

Causas de los problemas

- Insuficiente administración de requerimientos
- Comunicación ambigua e imprecisa
- Frágil arquitectura
- Complejidad impresionante
- Inconsistencia entre requerimientos, diseño e implementación
- Insuficiente proceso de pruebas
- Estimación subjetiva
- Proceso en cascada
- Cambio no controlados
- Insuficiente centralización

MEJORES PRÁCTICAS

- I. Desarrollo iterativo
- II. Administración de requisitos
- III. Uso de componentes arquitectónicos
- IV. Modelamiento visual (UML)
- V. Verificación continua de la calidad
- VI. Administración de cambios

I. Desarrollo iterativo (reducir riesgos y tiempo)

Un ciclo de desarrollo iterativo, disminuye los riesgos mas altos en la parte mas inicial.

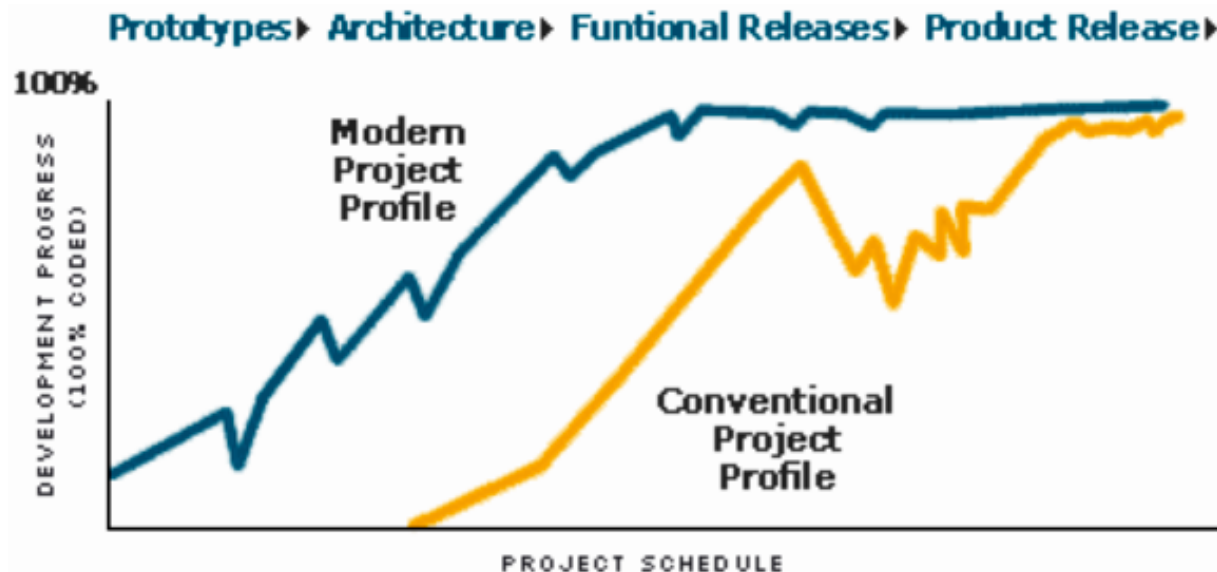
Cada iteración es un proceso de cascada en miniatura, iniciando con una reevaluación de requerimientos y terminando con una versión de software integrada y ejecutable.

Las ventajas principales son:

- ✓ Mitigación de riesgos
- ✓ Acomodación de cambios
- ✓ Alcanzar la más alta calidad
- ✓ Aumento de la reutilización

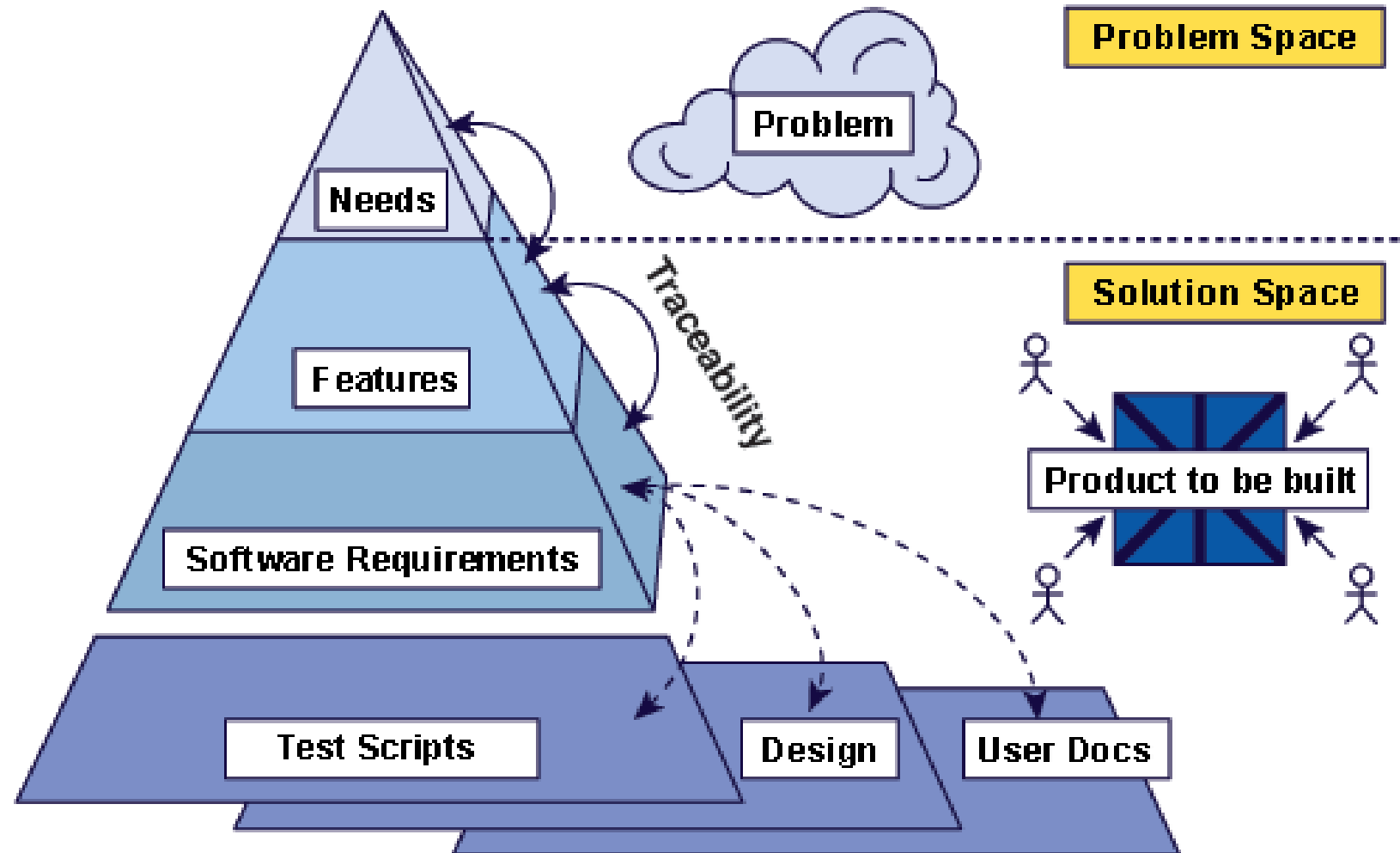
I. Desarrollo iterativo, continua...

Es necesario utilizar un acercamiento iterativo que permita un entendimiento incremental del problema a través de refinaciones sucesivas, creando de manera incremental una solución efectiva mediante múltiples iteraciones.



El desarrollo con iteraciones continuas reduce el riesgo de retrasos y sobrecostos de un proyecto

II. Administración de Requisitos (El primer paso del éxito del proyecto)



II. Administración de Requisitos, continua...

Estudios del Standish Group de 1997 al 2000 revelan que una de las principales causas para el fracaso de un proyecto de software es la mala (o ausencia de) administración de requerimientos.

Los principales problemas de un mal manejo de requerimientos son:

- ✓ Incapacidad para **manejar los cambios** en los requerimientos durante el desarrollo.
- ✓ Falta de **especificación detallada** de los requerimientos.
- ✓ Mala **organización y control** de requerimientos.
- ✓ Requerimientos **mal entendidos**.

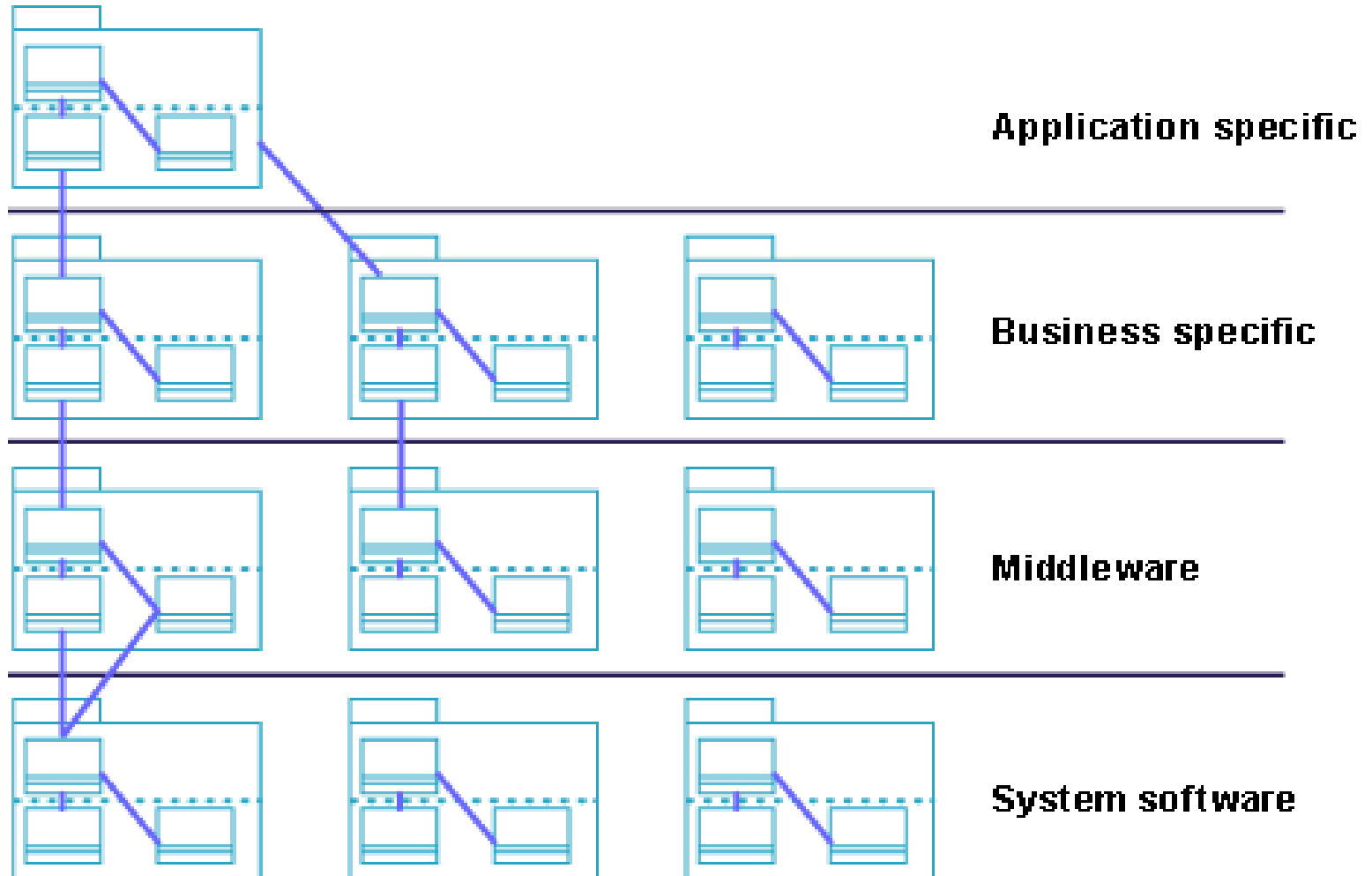
II. Administración de Requisitos, continua...

La administración de requerimientos comprende las actividades relacionadas con la definición, clasificación, asignación, seguimiento y control de los requerimientos durante todo el ciclo de vida de desarrollo de software. Es una metodología indispensable para el aseguramiento de la calidad de los productos, así como para el control y seguimiento de los proyectos.

Pasos en la Administración de Requerimientos:

- ✓ **Analizar** el problema
- ✓ **Entender** las necesidades del inversionista
- ✓ **Definir** el sistema
- ✓ **Administrar** el alcance del proyecto
- ✓ **Refinar** la definición del sistema
- ✓ **Administrar** el control de requerimientos

III. Componentes Arquitectónicos (La base para la reutilización de software)



III. Componentes Arquitectónicos, continua...

Uno de los peligros en sistemas grandes y complejos es que al hacer cambios a una parte de la aplicación se impacten otras partes con el "efecto dominó". Los equipos de desarrollo exitosos resuelven estas dependencias potenciales a través del uso de arquitecturas basadas en componentes. El ensamble de aplicaciones mediante módulos manejables crea arquitecturas resistentes y flexibles que evitan que los cambios al software conlleven esfuerzos masivos.

A medida que las organizaciones reutilizan componentes, reducen la cantidad de código a generar para cada aplicación. Así logran acelerar el tiempo de desarrollo y reducen la probabilidad de introducir errores al sistema.

III. Componentes Arquitectónicos, continua...

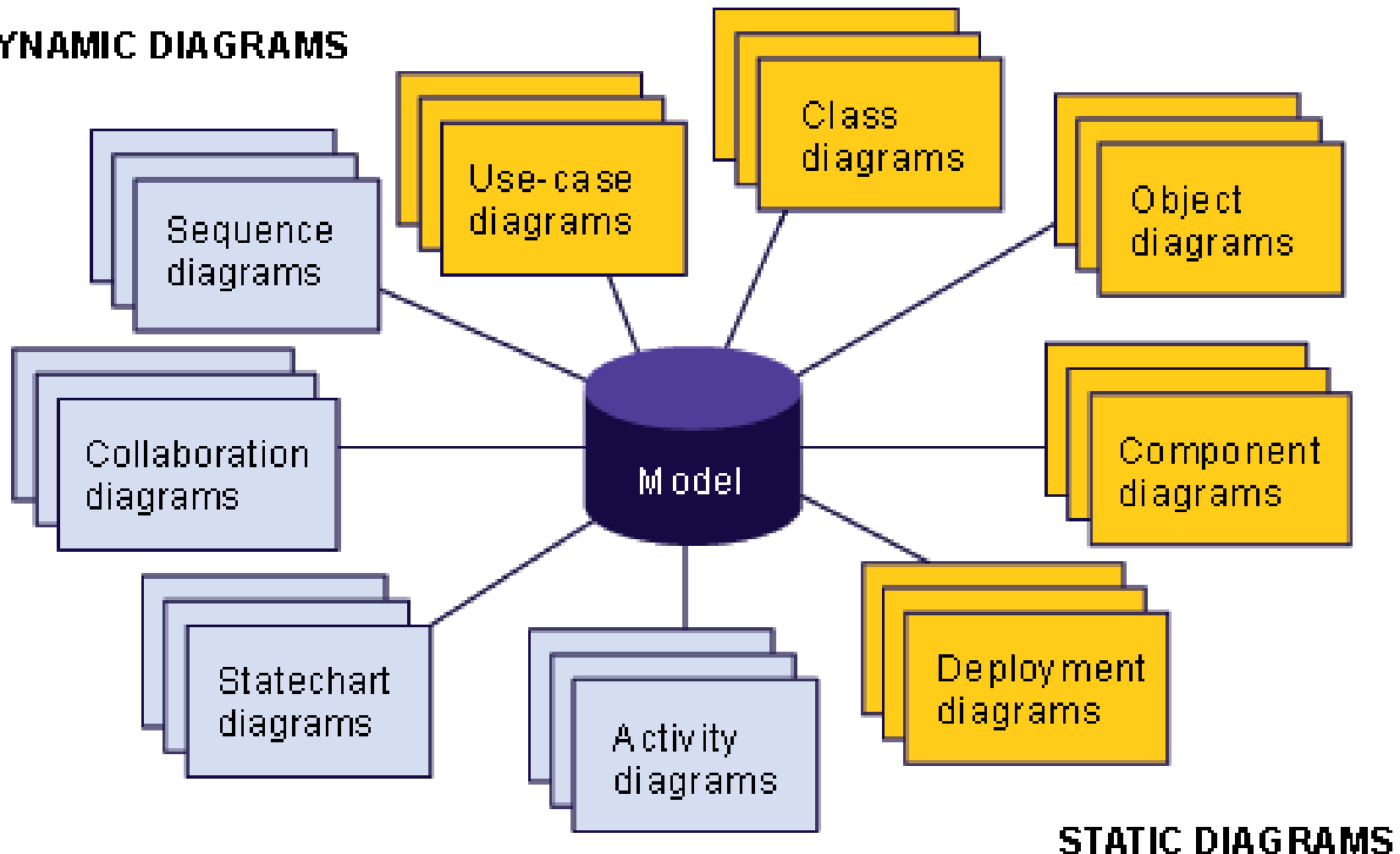
- ✓ **Facilita la reutilización**
 - Reutilización de componentes
 - Reutilización de arquitectura (frameworks)

- ✓ **Permite administrar mejor el proyecto**
 - Definición en primeras iteraciones
 - Planeación

- ✓ **Control intelectual**
 - Manejo de la complejidad
 - Mantiene la integridad

IV. Modelado Visual UML (Los planos del éxito)

DYNAMIC DIAGRAMS



IV. Modelado Visual UML, continua...

Para describir la complejidad de las operaciones actuales, los equipos de desarrollo necesitan una manera altamente descriptiva, intuitiva y precisa para modelar toda la funcionalidad. De lo contrario no serían capaces de comprender, especificar o documentar los sistemas que deben de construir. UML ([Unified Modeling Language](#)) es el lenguaje de modelado creado por Rational que se utiliza como el estándar para el desarrollo de software.

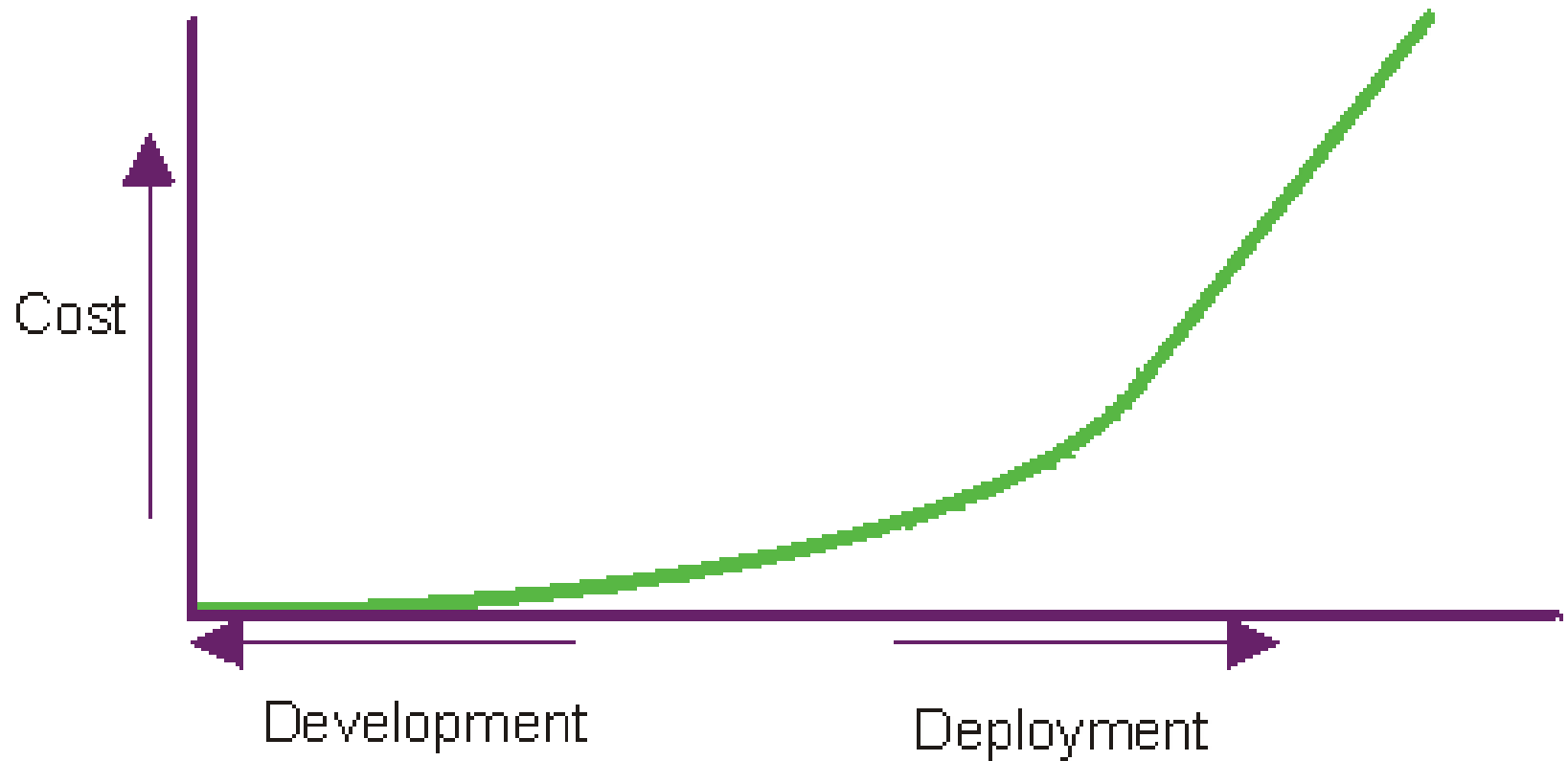
UML permite describir un sistema en diferentes niveles de abstracción, simplificando la complejidad sin perder información, para que tanto usuarios, líderes y desarrolladores puedan comprender claramente las características de la aplicación.

El [Rational Unified Process](#) describe cómo modelar visualmente aplicaciones para capturar la estructura y el comportamiento de la arquitectura y de los componentes.

IV. Modelado Visual UML, continua...

- ✓ Ayuda a entender la complejidad del sistema
- ✓ Captura la estructura y comportamientos de la arquitectura y componentes
- ✓ Mantiene la consistencia entre el diseño y su implementación
- ✓ Promueve un lenguaje común

V. Verificación de la Calidad (Construyendo con calidad de principio a fin)



V. Verificación de la Calidad, continua...

Es mucho más costoso **detectar y arreglar errores ya en producción que encontrarlos en etapas tempranas**. Los principales factores de rechazo de una aplicación son desempeño deficiente y baja confiabilidad. Por lo tanto, la calidad debe ser comprobada siempre con respecto a los requerimientos definidos previamente sobre funcionalidad, confiabilidad y desempeño, de la aplicación y del sistema).

La manera más efectiva de reducir los defectos en el desarrollo, es comenzar a realizar las **pruebas desde el inicio del ciclo de vida de desarrollo** y seguir probando incrementalmente toda la aplicación en cada iteración. Con este acercamiento, los defectos se corrigen tan pronto como se introducen en el producto y no al final, cuando su impacto es mayor.

V. Verificación de la Calidad, continua...

Adicionalmente, el conocer qué funcionalidades se han completado, es decir, codificado y probado, esto le permite al gerente del proyecto tener un mayor control sobre la aplicación y tener certeza del estado y grado de avance real. Asimismo, el probar la aplicación en cada iteración, promueve una **planeación y comunicación entre desarrolladores y testers**, lo cual crea una dinámica de equipo altamente enfocada, con recompensas substanciales.

La calidad no se debe tratar como una actividad adicional final realizada por un grupo independiente del equipo de desarrollo. Repetidamente se ha observado que los equipos de desarrollo que son exitosos, logran obtener un alto nivel de calidad mediante la utilización de herramientas de pruebas automatizadas.

V. Verificación de la Calidad, continua...

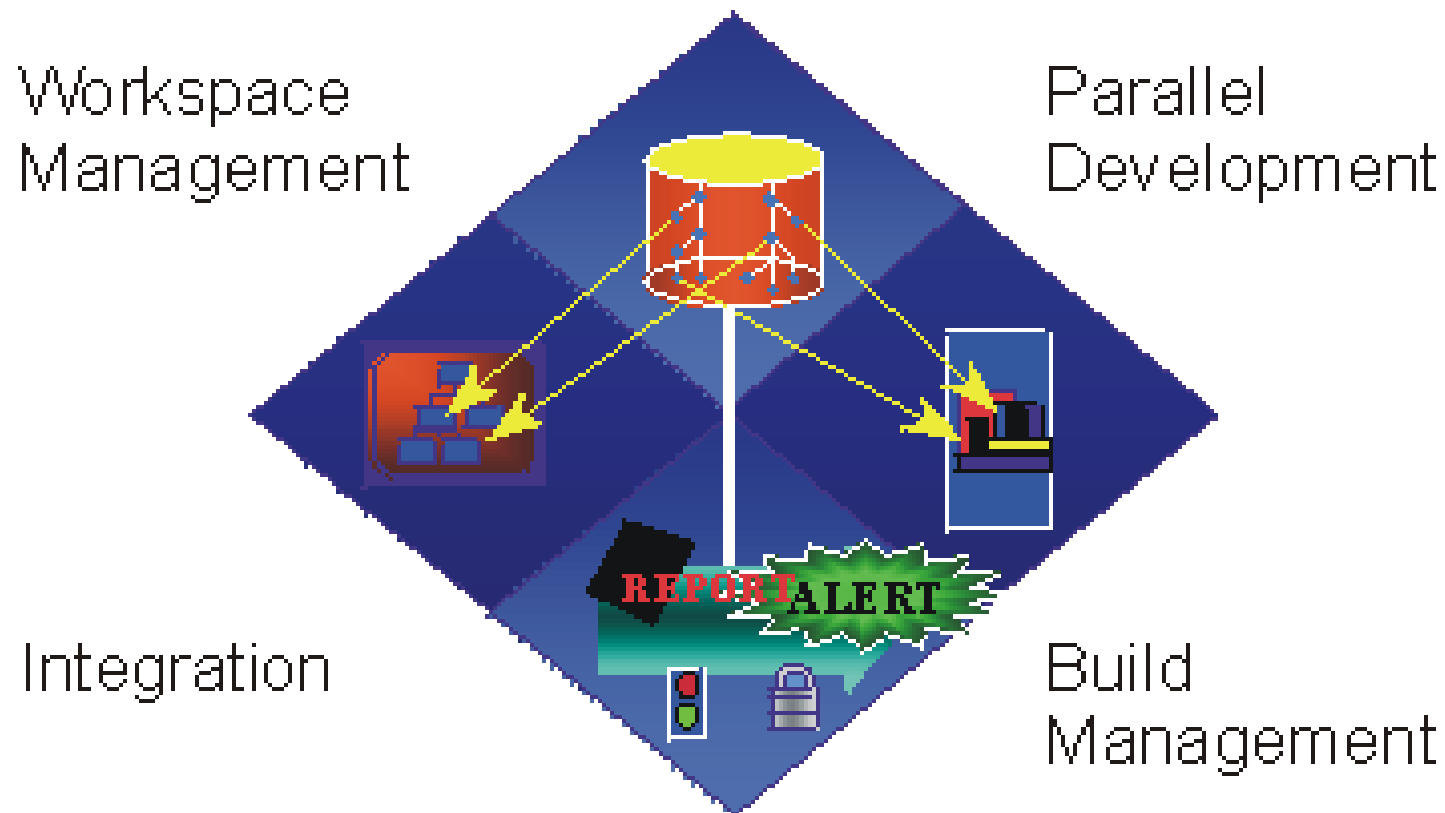
Calidad: la característica de demostrar el logro de la realización de un producto, que satisface los requerimientos convenidos Según lo estimado y conducido por un proceso conveniente.

Tres Dimensiones de la Calidad

- ✓ **Funcionalidad**: la aplicación hace lo requerido?
- ✓ **Confiabilidad**: la aplicación responde aceptablemente?
- ✓ **Rendimiento**: la aplicación rinde bajo cargas de producción?

VI. Control de Cambios

(Controlar el proyecto y eliminar los retrasos)



VI. Control de Cambios, continua...

El trabajo efectivo requiere de una administración formal de los cambios. Cuando se cuenta con una administración de cambios del software realmente efectiva se logra que:

- ✓ Los equipos de desarrollo pueden hacer liberaciones a tiempo, en presupuesto y con una calidad predecible.
- ✓ Los líderes de proyecto conozcan en todo momento el estado y avance del desarrollo y tengan certeza en el tiempo de liberación.
- ✓ Los testers sepan cuando una nueva construcción requiere ser probada y qué mejoras o correcciones debe presentar.
- ✓ Los desarrolladores manipulen y controlen con orden y seguridad sus enormes colecciones de archivos y componentes diferentes para cada aplicación.

VI. Control de Cambios, continua...

- ✓ Coordinación de artefactos y actividades
- ✓ Coordinación de iteraciones y releases
- ✓ Control de cambios de software
- ✓ Control de las versiones de los artefactos

ESTRUCTURA DEL RUP

F. Trabajo Procesos

Modelación de Negocios

Requerimientos

Análisis y Diseño

Implementación

Prueba

Desarrollo

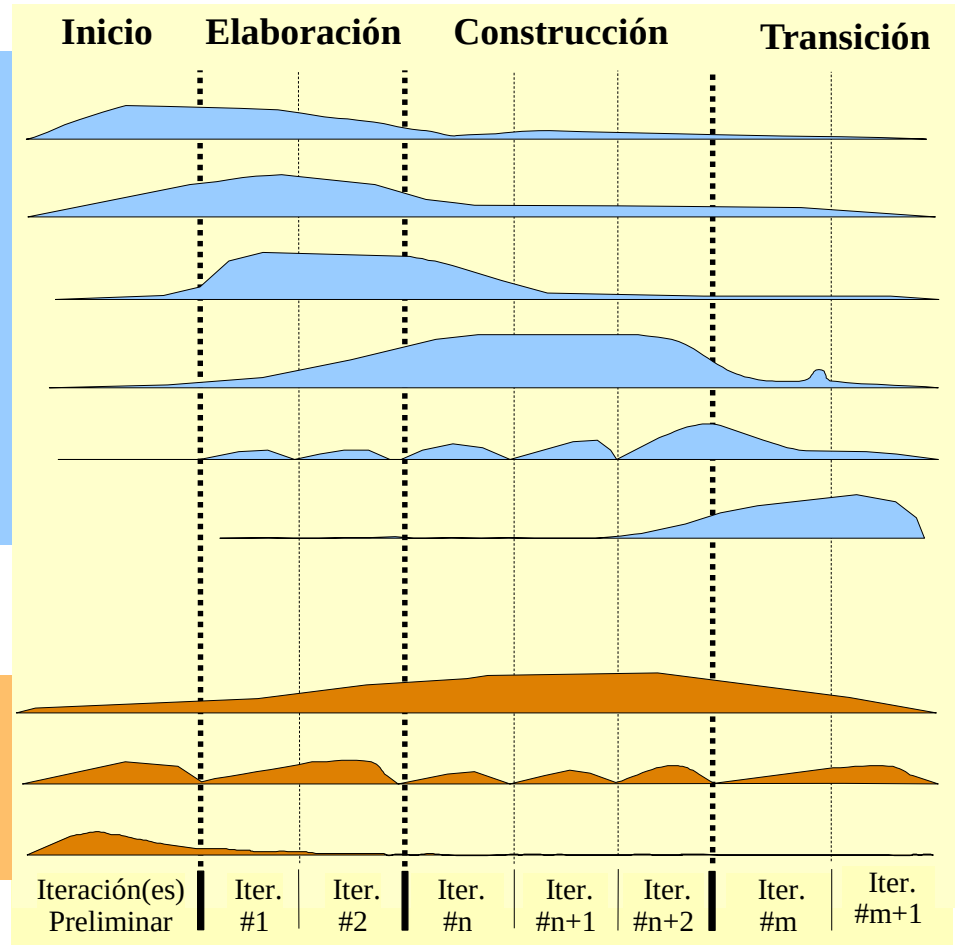
F. Trabajo Soporte

Admin. Configuración

Administración

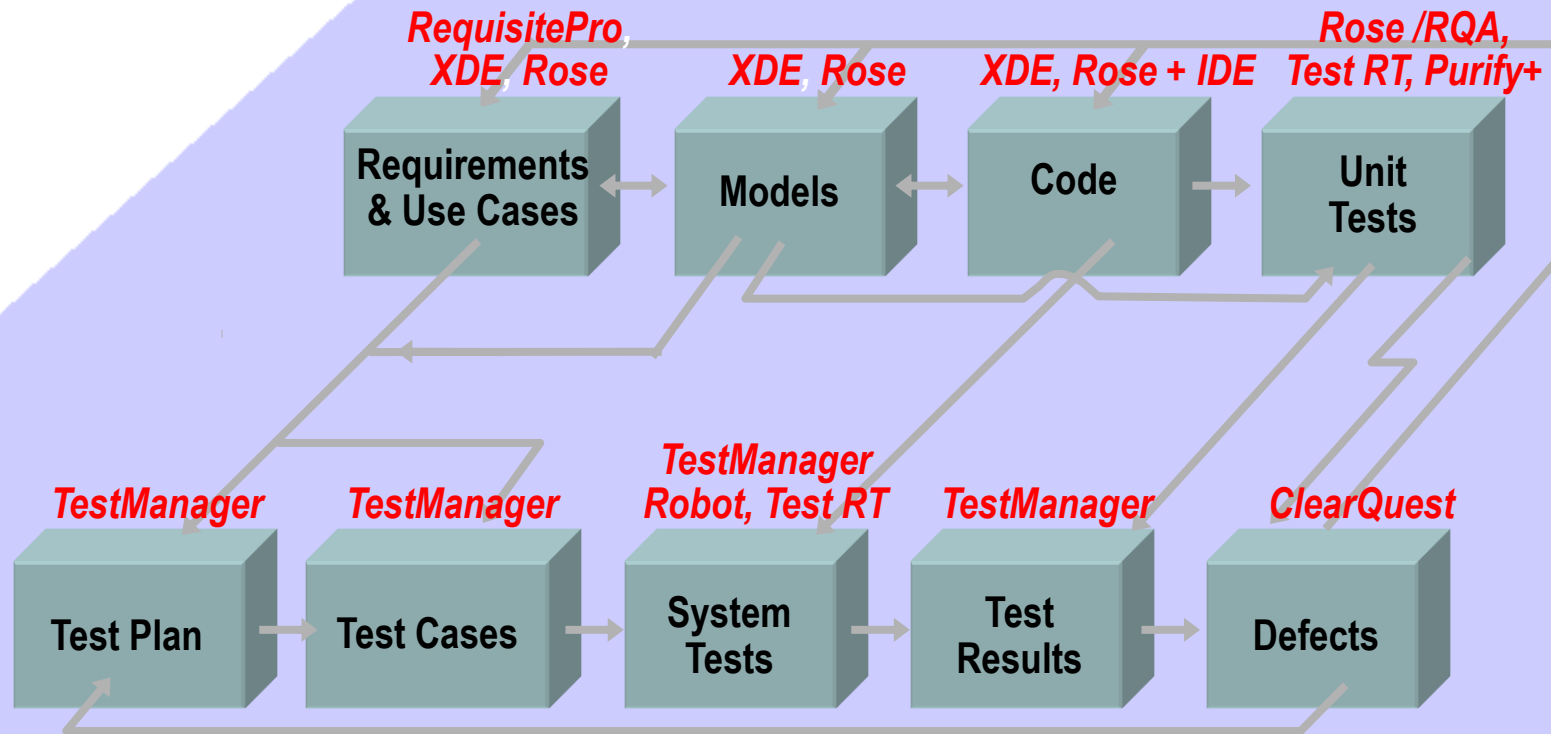
Ambiente

Fases



Iteraciones

La Plataforma Rational



Common Process and Guidance – *Rational Unified Process, Rational Developer Network*

Progress Metrics and Reporting – *SoDA, ProjectConsole*

Software Configuration Management – *ClearCase, ClearQuest*

Estructura del RUP, continua...

- El proceso puede describirse en dos dimensiones, o a lo largo de dos ejes:
 - ✓ **El eje horizontal** representa *tiempo* y muestra el aspecto dinámico del proceso, expresado en términos de *ciclos*, *fases*, *iteraciones*, y *metas*.
 - ✓ **El eje vertical** representa el aspecto estático del proceso; como está descrito en términos de *actividades*, *artefactos*, *trabajadores* y *flujos de trabajo*.

Taller Rup - UML

FASES DEL RUP:

Cada fase concluye con un HITO. (Toma Decisiones)

- ✓ **Inicio** → Define el alcance del proyecto.
- ✓ **Elaboración** → Plan del proyecto, especificación de características, arquitectura base.
- ✓ **Construcción** → Construir el producto.
- ✓ **Transición** → Transición del producto a la comunidad del usuario.



I. Fase de Inicio (los objetivos del ciclo de vida)

Objetivos específicos:

- Establecer el alcance y fronteras del software del proyecto, incluyendo la **visión operacional**, criterio de aceptación, qué se espera que esté en el producto y qué no.
- Discriminar los casos de uso críticos del sistema, los **escenarios primarios de operación** que dirigirán las principales decisiones de diseño.
- Mostrar al menos una **arquitectura candidata** para alguno de los escenarios primarios.
- **Estimar el coste global y planificación para el proyecto completo** (estimaciones más precisas se obtendrán en la fase siguiente).
- Estimar los **riesgos potenciales**.
- Preparar el **ambiente de soporte** al proyecto.

I. Fase de Inicio, continua...

Actividades:

- Formular el alcance del proyecto.
- Planificar y preparar el caso de negocio.
- Sintetizar una arquitectura candidata.
- Preparar el ambiente del proyecto: evaluar el proyecto y la organización, seleccionar las herramientas, decidir qué partes del proceso mejorar.

Hito principal de la fase:

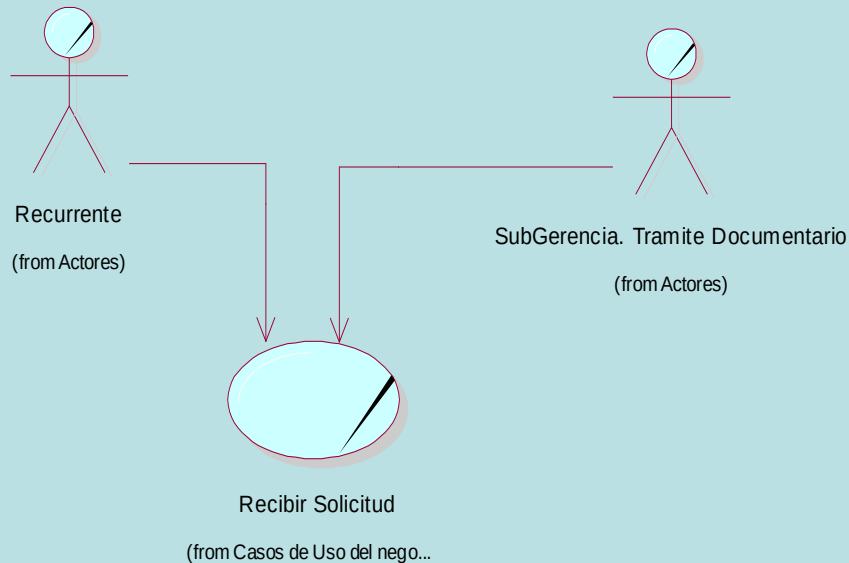
Establecer el ámbito del producto, identificando los principales riesgos, y adquirido los elementos de juicio suficientes para decidir sobre la viabilidad del proyecto.

I. Fase de Inicio – resumen -, continua...

- Propósito
 - ✓ Establecer casos de negocios para un nuevo sistema o una actualización importante en el sistema existente. Se identifican todas las entidades externas con las que se trata (actores).
 - ✓ Especificar el alcance del proyecto
- Resultado
 - ✓ Una visión general del proyecto: requerimientos, características y restricciones principales
 - ✓ Un Modelo del Negocio: contexto, criterios de éxito y pronóstico financiero
 - ✓ Un MCU inicial y modelo del dominio (10-20%)
 - ✓ Identificación de riesgos, estimación de recursos y Plan de c/u de las Fases con sus Hitos.

Taller Rup - UML

I. Fase de Inicio, continua...



Caso de Negocios: modelar la empresa (como funciona la empresa a la que se le va a desarrollar el software).

Bussines Worker: Actor Interno (del negocio)

Bussines Actor : Actor Externo (del negocio)

Bussines Use Case: Caso de Uso (del negocio)

II. Fase de Elaboración (la arquitectura del ciclo de vida)

Objetivos específicos:

Asegurar que la arquitectura, requisitos y planes son lo suficientemente estables y los riesgos han sido mitigados para poder determinar los costes y la planificación necesaria para completar el desarrollo.

Solucionar todos los riesgos significativos para la arquitectura del proyecto.

Establecer la línea base de la arquitectura obtenida después de tratar los escenarios más significativos para la arquitectura.

Producir un prototipo progresivo de componentes con calidad para la producción.

Establecer el ambiente de soporte para el proyecto. Esto incluye crear los planes de desarrollo, preparar las plantillas de los documentos, instrucciones, y herramientas.

II. Fase de Elaboración, continua...

Actividades:

Definir, validar y añadir la arquitectura a la línea base.

Refinar la Visión basándose en información nueva obtenida durante la fase.

Crear y añadir a la línea base planes de iteración detallados para la fase de construcción.

Refinar los planes de desarrollo y ponerlos en práctica en el ambiente de desarrollo. Refinar la arquitectura y seleccionar componentes.

Hito principal de la fase:

Obtener una línea base de la arquitectura del sistema, capturar la mayoría de los requisitos y reducir los riesgos principales así como permitir la escalabilidad del equipo del proyecto durante la fase de construcción.

II. Fase de Elaboración, - en resumen - continua...

- Propósito
 - ✓ Analizar el dominio del problema
 - ✓ Establecer una buena arquitectura
 - ✓ Lidiar con los riesgos más altos del proyecto
 - ✓ Desarrollar un Plan de Proyecto
- Resultado
 - ✓ Un modelo del dominio y de MCU (80% completo)
 - ✓ Otros requerimientos no funcionales o que no estén asociados con un CU específico
 - ✓ Descripción de la Arquitectura del SW
 - ✓ Un prototipo ejecutable de la arquitectura
 - ✓ Una lista de riesgos revisada
 - ✓ Plan de desarrollo para el resto del proyecto
 - ✓ Manual de Usuarios (preliminar).

III. Fase de Construcción (Capacidad operativa inicial)

Objetivos específicos:

Minimizar los costes de desarrollo **optimizando los recursos** y evitando cambios innecesarios que resulten en desechar o modificar trabajo ya realizado.

Obtener **versiones útiles** (alfa, beta, y otras entregas de prueba).

Completar el análisis, **diseño, desarrollo y prueba** de toda la funcionalidad requerida.

Desarrollar de forma iterativa e incremental un producto completo que esté listo para su transición hacia la comunidad de usuarios. Esto implica detallar los casos de uso restantes y otros requisitos así como completar el diseño, implementación y prueba del software.

Decidir si el **software, sitio y usuarios** están listos para la instalación de la aplicación.

III. Fase de Construcción, continua...

Actividades:

Administración de recursos, control y optimización del proceso.

Desarrollo y prueba completa de los componentes utilizando el criterio de evaluación definido.

✓

Evaluación de las entregas del producto utilizando los criterios de aceptación de la Visión.

Hito principal de la fase:

Desarrollo del sistema con calidad de producción, y puede entonces prepararse para la entrega al equipo de transición. En esta fase, toda la funcionalidad ha sido implementada, y completadas las pruebas para el estado alfa de la aplicación.

III. Fase de Construcción, - en resumen - continua...

- Propósito
 - ✓ Desarrollar incrementalmente el producto (SW completo) el cual será probado y transferido al usuario
- Productos
 - ✓ Un modelo completo de diseño y casos de uso
 - ✓ Liberaciones de productos ejecutables de funcionalidad incremental
 - ✓ Documentación de usuario
 - ✓ El producto de software integrado y corriendo en la plataforma adecuada.
 - ✓ Una liberación “beta” del producto

IV. Fase de Transición, continua...

Objetivos específicos:

Realizar pruebas para validar el nuevo sistema con las expectativas de los usuarios. Realizar pruebas y la operación en paralelo al sistema anterior que se está reemplazando.

Entrenamiento de usuarios y encargados del mantenimiento.

Actividades de corrección de errores, mejoras en el funcionamiento y rendimiento y usabilidad.

Evaluación de la línea base de la instalación con la visión completa y criterios de la aceptación del producto.

Lograr el consenso de los involucrados en que la línea base esta completa y consistente con el criterio de evaluación de la visión.

Ingeniería específica de instalación: comercialización y producción de los paquetes, etc.

IV. Fase de Transición, continua...

Actividades:

- ✓ Ejecutar los planes de instalación.
- ✓ Completar el material de soporte de los usuarios finales.
- ✓ Pruebas del producto en el sitio de desarrollo.
- ✓ Preparar la entrega de esta versión del producto.
- ✓ Recoger la retroalimentación de los usuarios.
- ✓ Hacer los ajustes al producto basándose en la retroalimentación de los usuarios.
- ✓ Hacer disponible el producto para los usuarios finales.

Hito principal de la fase:

Consiste en decidir si los objetivos se cumplieron y si debe comenzarse otro ciclo de desarrollo. Es el resultado de la revisión y aceptación por parte del cliente de los productos y subproductos que le han sido entregados.

IV. Fase de Transición, - en resumen - continua...

Propósito

- Hacer la transición final del producto de SW al usuario
- Ver que el usuario quede satisfecho según sus requerimientos

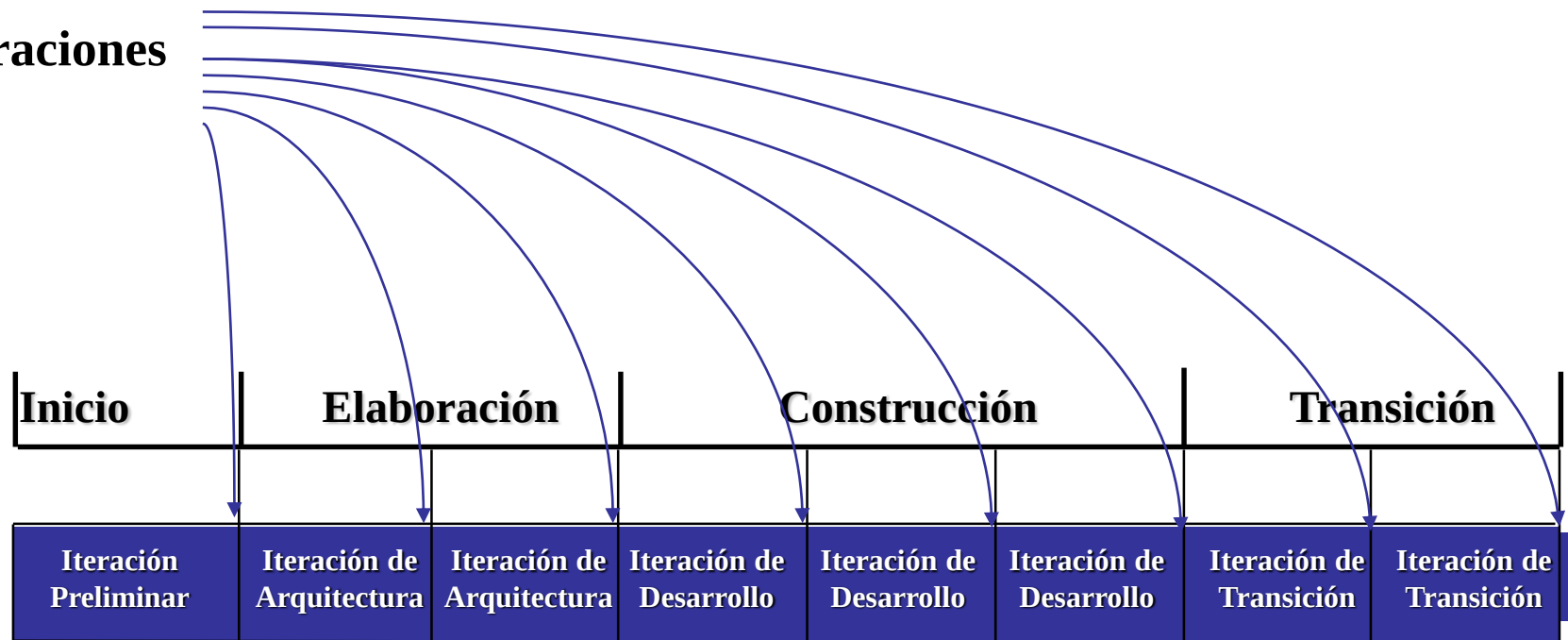
Productos:

- ✓ Liberaciones ejecutables de producto (comparación paralela con sistemas antiguos)
- ✓ “Pruebas beta” para validar el nuevo sistema vs. las expectativas del usuario
- ✓ Manuales de usuario actualizados
- ✓ Documentación de desarrollo actualizada
- ✓ Entrenamiento de usuarios
- ✓ Distribución del producto.

ITERACCION

- Cada fase en RUP puede descomponerse en iteraciones. Una *iteración* es un ciclo de desarrollo completo dando como resultado una entrega de producto ejecutable (interna o externa)

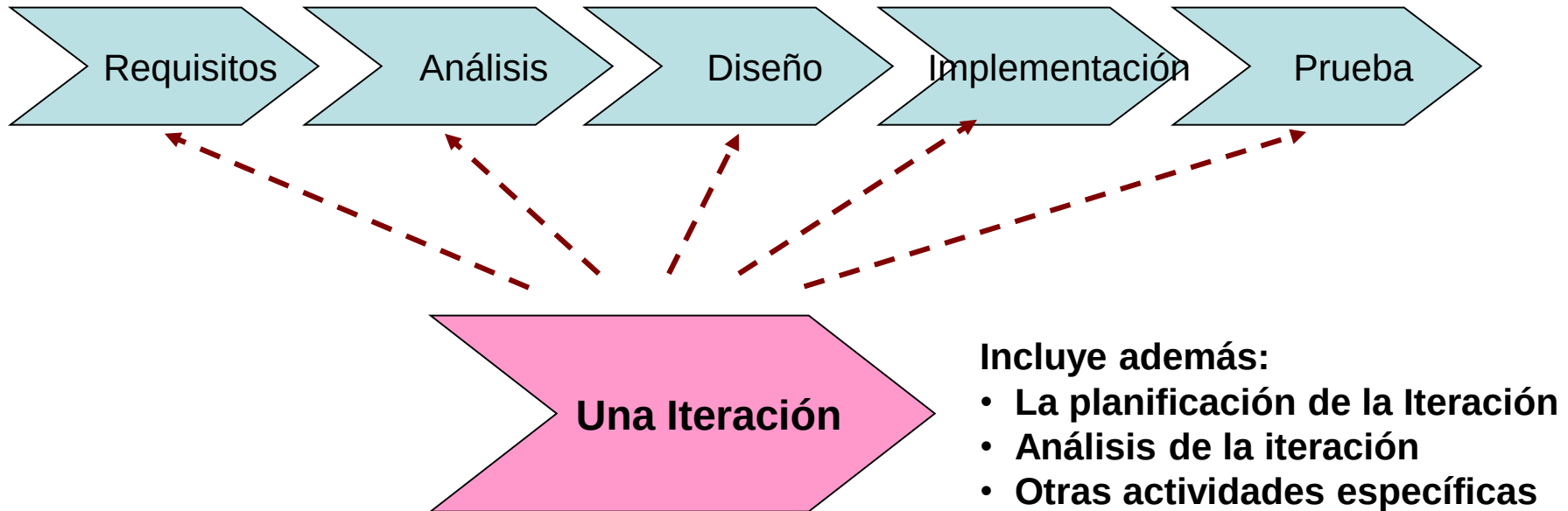
Liberaciones



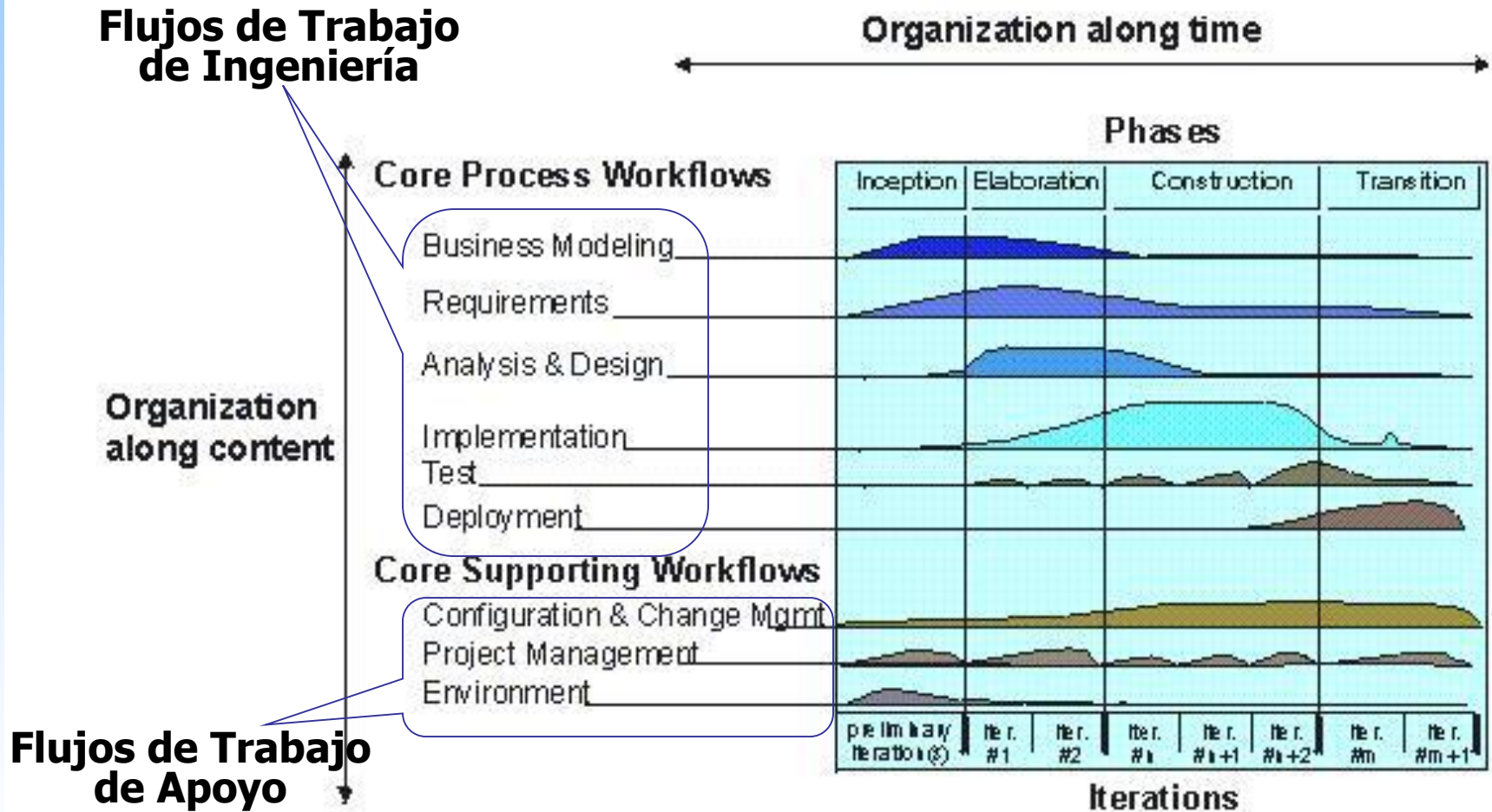
Taller Rup - UML

Iteración, continua...

- Es un mini proyecto que pasa por 5 flujos fundamentales de trabajo.
- Cada iteración se analiza cuando se termina, para poder iniciar otra.



FLUJOS DE TRABAJO



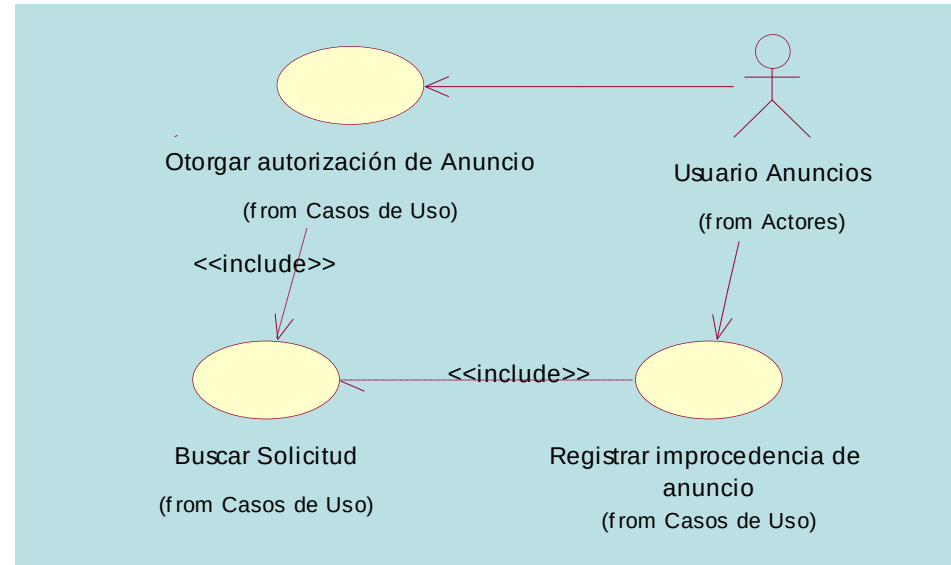
I. Modelo del Negocio

Para conseguir sus objetivos, una empresa organiza su actividad por medio de un **conjunto de procesos de negocio**. Cada uno de ellos se caracteriza por una **colección de datos** que son producidos y manipulados mediante un **conjunto de tareas**, en las que ciertos **agentes** (por ejemplo, trabajadores o departamentos) participan de acuerdo a un flujo de trabajo determinado. Además, estos procesos se hallan sujetos a un conjunto de **reglas de negocio**, que determinan la estructura de la información y las políticas de la empresa.

La finalidad del modelado del negocio, es describir cada proceso del negocio del cliente, especificando sus datos, actividades (o tareas), roles (o agentes) y reglas de negocio.

II. Requerimiento

- Los desarrolladores y clientes deben acordar qué es lo que el sistema debe hacer:
 - ✓ Relevar requerimientos
 - ✓ Documentar funcionalidad y restricciones
 - ✓ Documentar decisiones
 - ✓ Identificar actores
 - ✓ Identificar CU



- Los casos de uso describen la funcionalidad.
- Los requerimientos no funcionales se incluyen en una especificación complementaria.
- Permite definir la interfaz de usuario del sistema enfocándose en sus necesidades.

III. Análisis y Diseño

- Descripción de cómo se implementará el sistema: un plano
- Debe:
 - Ejecutar las tareas y funciones descritas en los casos de uso
 - Satisfacer todos los requerimientos
 - Flexible a cambios
- El diseño se centra en la noción de arquitectura.
- Diseñar y validar la arquitectura es una tarea esencial.
- El modelo de diseño consta de:
 - Clases estructuradas en paquetes
 - Diseños de subsistemas con interfaces definidas (componentes)
 - Forma de colaboración entre las clases.

Se integrarán los resultados producidos por los programadores individuales en un solo sistema ejecutable.

IV. Implementación

- **Propósito:**
 - ✓ Definir la organización del código
 - ✓ Implementar clases y objetos en forma de componentes (fuente, ejecutables, etc.)
 - ✓ Probar las componentes desarrolladas
 - ✓ Integrar las componentes en un sistema ejecutable

Permite definir la **organización del código en términos de subsistemas y capas**. Convertir los elementos del diseño en elementos de implementación (fichero fuentes, binarios, ejecutables, y otros). Realizar pruebas de unidad a los componentes desarrollados.

V. Pruebas

- **Propósito:**
 - ✓ Verificar la interacción entre los objetos
 - ✓ Verificar la integración apropiada de componentes
 - ✓ Verificar que se satisfacen los requerimientos
 - ✓ Identificar los defectos y corregirlos antes de la instalación
- RUP propone probar las componentes desde el principio:
 - ✓ **Confiabilidad, funcionalidad y performance**
- Las pruebas de regresión son importantes en desarrollos iterativos.
- Rational tiene herramientas para automatizar algunas pruebas.

Su objetivo es encontrar y documentar defectos en la calidad del software

VI. Distribución

- Producir un producto y hacerlo llegar a sus usuarios finales.
- Incluye varias actividades:
 - ✓ Producir un “release”
 - ✓ Empaquetar el software
 - ✓ Distribuir el software
 - ✓ Instalar el software
 - ✓ Apoyar a los usuarios
- A veces también incluye:
 - ✓ Realizar pruebas beta
 - ✓ Migración de datos
 - ✓ Aceptación formal
- La mayor parte de la distribución ocurre durante la transición.
- Este es uno de los flujos de trabajo menos documentados en RUP.

FLUJO DE SOPORTE

I. Administración de Configuración y Cambios

Controla los cambios y mantiene la integridad de los artefactos del proyecto. Esta disciplina se encarga de: Identificar los elementos de configuración, restringir y auditar los cambios a esos elementos, y definir y gestionar las configuraciones de esos elementos.

- Algunos problemas habituales:
 - ✓ Actualizaciones simultáneas
 - ✓ Múltiples versiones
- RUP da guías para:
 - ✓ Desarrollos en paralelo
 - ✓ Automatizar la construcción
 - ✓ Administrar defectos

II. Administración de Proyectos

Es el arte de balancear objetivos en competencia, gestionar los riesgos y sobreponerse a las restricciones para crear con éxito un producto que satisfaga las necesidades tanto de los clientes como de los usuarios finales.

- RUP incluye:
 - ✓ Un framework para manejo de proyectos de software
 - ✓ Guías para planificación, provisión de personal, ejecución y monitoreo de planes
 - ✓ Un framework para manejar riesgos

III. Ambiente

Proporciona a la organización el entorno de desarrollo de software apropiado, que contendrá las herramientas de desarrollo y del proceso, plantillas, documentos, convenciones a seguir, y cualquier otro elemento necesario para llevar adelante con éxito el desarrollo del proyecto.

- Ambiente y herramientas de desarrollo que harán posible llevar a cabo el proyecto.
- RUP guía en la configuración de un ambiente de proceso apropiado a cada proyecto.